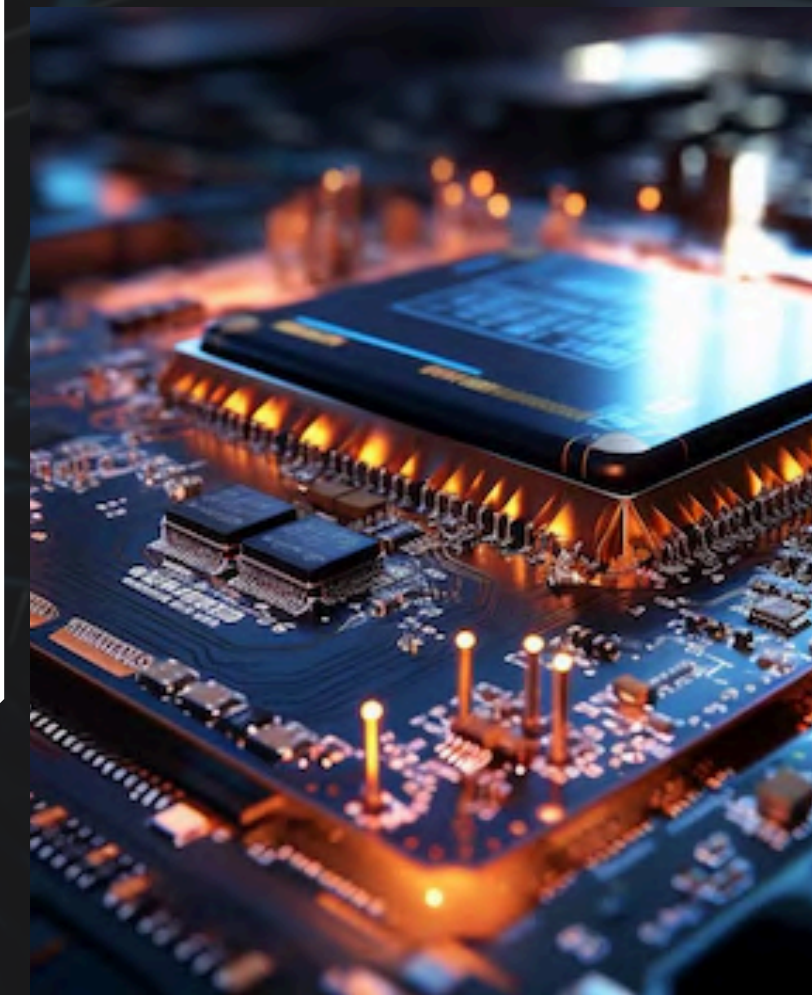




STM32 BASED GPS CLOCK

(An Accurate, Internet-Independent, Timekeeping
Solution)



Arpit Raj
Abhinav Maurya

BILL OF MATERIALS

Component	Quantity	Source
STM32 Nucleo F411RE	1	ECE Labs, IITD
NEO-6M GPS Module	1	ECE Labs, IITD
DS3231 RTC Module	1	Graylogix
TM1637 4-Digit 7-Segment LED Display	1	Graylogix
Jumper Wires	-	Local/ECE Labs

DS 3231 RTC MODULE PURCHASE LINK: [HTTPS://WWW.GRAYLOGIX.IN/PRODUCT/DS3231-AT24C32-IIC-MODULE-PRECISION-CLOCK-MODULE-WITH-CR2032-BATTERY](https://www.graylogix.in/product/ds3231-at24c32-iic-module-precision-clock-module-with-cr2032-battery)

7 SEGMENT LED DISPLAY PURCHASE LINK: [HTTPS://WWW.GRAYLOGIX.IN/PRODUCT/TM1637-LED-DISPLAY-MODULE-7-SEGMENT-4-BITS-0-36INCH](https://www.graylogix.in/product/tm1637-led-display-module-7-segment-4-bits-0-36inch)

PROBLEM STATEMENT

Accurate time synchronization is critical in many real-world applications like railway stations, industrial automation, public clocks, and embedded systems.

Traditional clocks may drift over time, and Internet-based synchronization is not always feasible, especially in remote or secured areas.

There is a strong need for a highly accurate, standalone, GPS-synchronized clock with an intelligent fallback mechanism to ensure continuous and reliable timekeeping even during GPS signal loss.



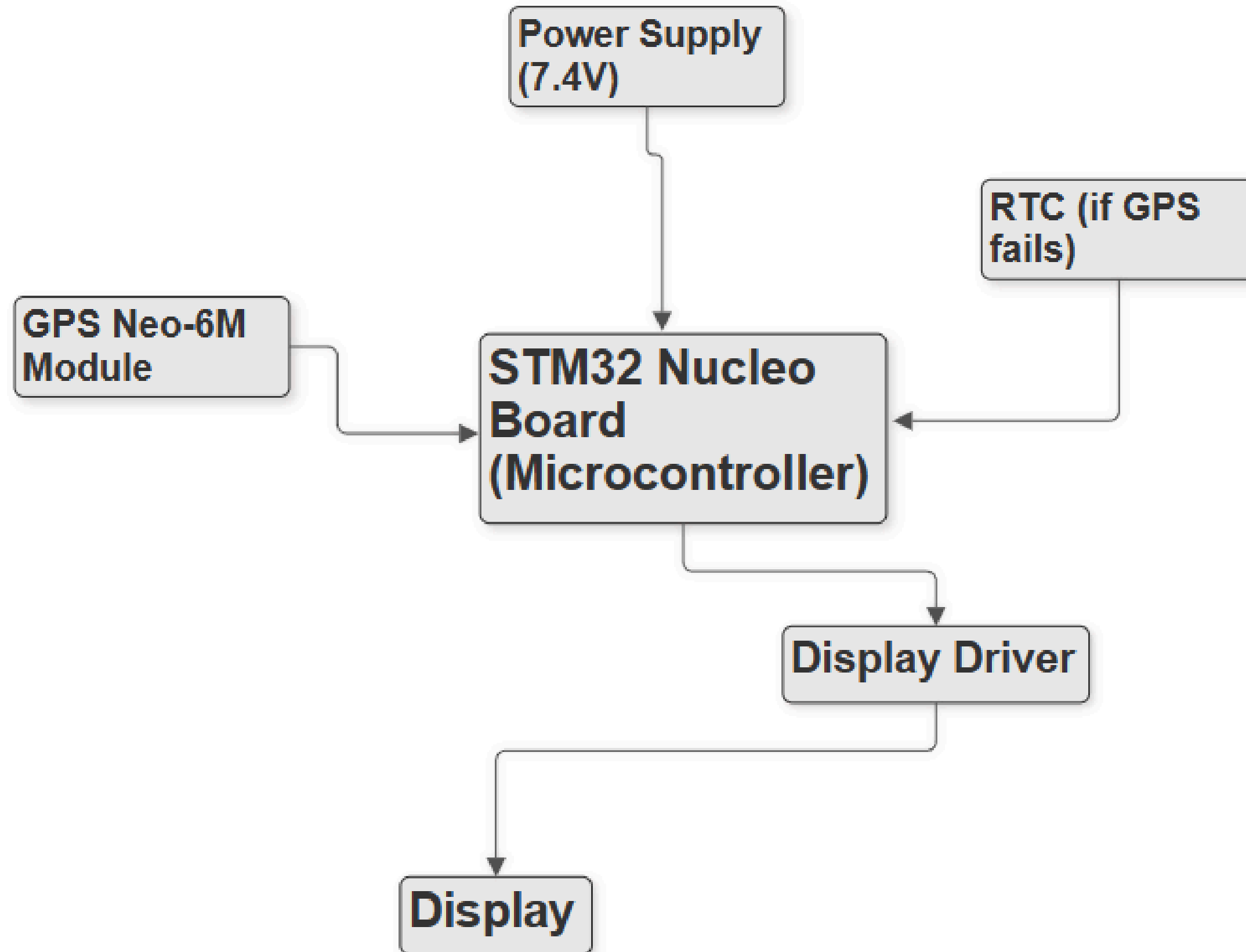
PROJECT OBJECTIVE

This project aims to develop a precise GPS-based digital clock using the STM32 platform, which can maintain time even during GPS signal outages by switching to a Real-Time Clock (RTC) module. The design focuses on:

- High Accuracy: Using GPS as the primary time source.
- Robustness: RTC fallback to maintain continuous operation.
- Low Power: Optimized for low power consumption using STM32's capabilities.
- User-Friendly Display: Via a bright 4-digit 7-segment LED module.



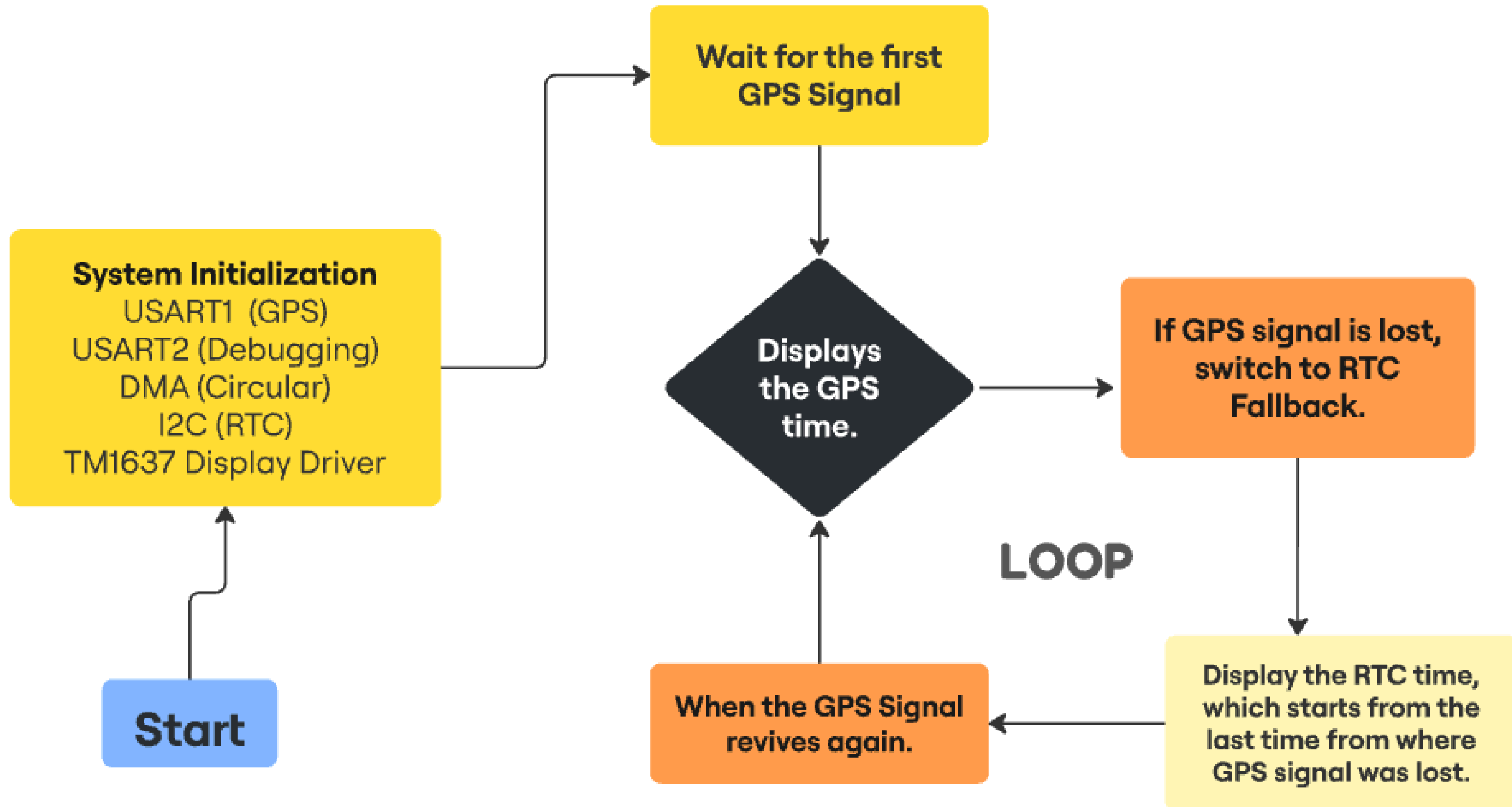
BLOCK DIAGRAM



BLOCK DIAGRAM INFORMATION

Component	Type	Interface	Power	Role
STM32 Nucleo F411RE	Controller	UART, I2C, GPIO	3.3V	Central processing unit
GPS Module (NEO-6M)	Input	UART	3.3V / 5V	Primary time source
RTC Module (DS3231)	Input (Fallback)	I2C	3.3V	Backup time source
Display Driver (TM1637)	Output	2-Wire Serial	5V	Controls the display
Display (4-Digit 7-Segment)	Output	Driven by TM1637	5V	Visual time display
Power Supply	Power Input	Direct	7.4V → Regulated	Provides system power

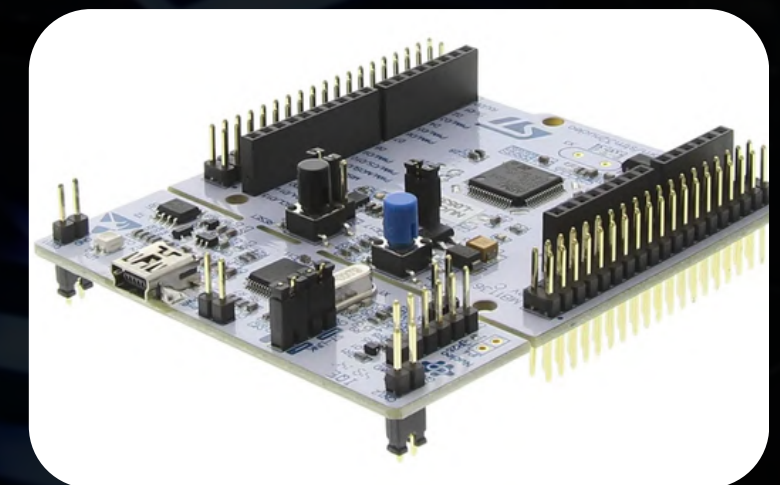
FLOWCHART



SYSTEM OVERVIEW

The GPS Clock system consists of:

- **GPS Module (NEO-6M):** Fetches UTC time via satellite.
- **STM32 Nucleo F411RE:** Core processing unit for GPS parsing, RTC handling, and display control.
- **DS3231 RTC Module:** Maintains time during GPS outages with battery backup.
- **TM1637 LED Display:** Provides real-time, user-friendly time display.
- **Fallback Logic:** Automatically switches to RTC in the absence of GPS lock.



SOFTWARE ENVIRONMENT



STM32 CubeIDE
for development
and debugging.



HAL Libraries for
UART (GPS), I2C
(RTC), and TM1637
display control.



UART GPS Parsing:
Extracts \$GPRMC or
\$GPGGA sentences
for accurate time.



RTC I2C Communication:
Reads and writes to
DS3231 registers.



TM1637 Driver: Custom or
library-based code to
interface the display.

KEY FEATURES



- **High Precision:** Sub-second synchronization using GPS NMEA sentences.
- **Automatic Fallback:** Intelligent switch to RTC in case of GPS signal loss.
- **Real-Time Display:** Easy-to-read TM1637 4-digit display.
- **Internet-Independent:** Does not require NTP servers or online sync.
- **Low Power:** Efficient STM32 MCU operation.
- **Modular Design:** Components can be reused or upgraded easily.
- **Highly Scalable:** Can be extended to date display, alarm functionalities, bilingual or large public clocks.



WORKING PRINCIPLE

- System powers on and initializes GPS, RTC, and display.
- GPS provides real-time UTC time to STM32 via USART.
- STM32 displays GPS time on the 7-segment display.
- If GPS signal is lost, STM32 automatically switches to RTC for timekeeping.
- When GPS is restored, system resynchronizes RTC and returns to GPS mode.
- The cycle continues, always prioritizing GPS if available.

Time = 17:20:51	Lat = 2832.893555	Long = 7716.487793
Time = 17:20:53	Lat = 2832.892578	Long = 7716.488281
Time = 17:20:54	Lat = 2832.892090	Long = 7716.488281
Time = 17:20:56	Lat = 2832.892090	Long = 7716.488281
Time = 17:20:57	Lat = 2832.891846	Long = 7716.488281
Time = 17:20:58	Lat = 2832.892334	Long = 7716.487793
Time = 17:21:00	Lat = 2832.892090	Long = 7716.487305
Time = 17:21:01	Lat = 2832.891357	Long = 7716.487305
Time = 17:21:03	Lat = 2832.891357	Long = 7716.486816
Time = 17:21:04	Lat = 2832.891113	Long = 7716.486328
Time = 17:21:06	Lat = 2832.891602	Long = 7716.485840
Time = 17:21:07	Lat = 2832.891846	Long = 7716.485840
Time = 17:21:09	Lat = 2832.892334	Long = 7716.485352
Time = 17:21:10	Lat = 2832.892578	Long = 7716.484863
Time = 17:21:12	Lat = 2832.893311	Long = 7716.484375
Time = 17:21:13	Lat = 2832.893311	Long = 7716.484375
Time = 17:21:15	Lat = 2832.893311	Long = 7716.483887
Time = 17:21:16	Lat = 2832.893311	Long = 7716.483887
Time = 17:21:18	Lat = 2832.893311	Long = 7716.483398
Time = 17:21:19	Lat = 2832.893311	Long = 7716.482910
Time = 17:21:21	Lat = 2832.893311	Long = 7716.482422
Time = 17:21:22	Lat = 2832.893799	Long = 7716.481934
Time = 17:21:24	Lat = 2832.894043	Long = 7716.481445

MAJOR CHALLENGES FACED

GPS Didn't Work Indoors

One of the first problems we faced was that the GPS simply didn't work inside the lab. No matter what we tried, the GPS couldn't catch any satellite signals indoors. Because of this, we had to keep taking our setup outside again and again just to test it, which was a bit tiring and also slowed us down.

Continuous GPS Data

While working with the USART communication, we faced challenges because the GPS was sending data continuously and asynchronously. It was difficult to manage this live incoming stream while the STM32 was performing other tasks simultaneously. To resolve this, we configured DMA in circular mode using CubeIDE so that the GPS data could be continuously received and stored efficiently without burdening the CPU.

Buffer Overflows and Edge Cases

At first, our system sometimes crashed because of buffer overflows. The GPS was sending more data than we had space for. This was especially tricky when we were converting timezones and handling cases like when the day changed at midnight. So, we added buffer size checks to make sure we never crossed the memory limit.

I2C Setup for RTC

The RTC uses I2C communication, which we had to set up separately in CubeIDE. It took us some time to understand how to properly configure I2C for the RTC module because it was different from how we set up the GPS (which used USART).

GPS Data and Display Format Mismatch

The GPS was sending time in one format and the seven-segment display needed it in another format. So, we had to write extra code to convert the GPS data properly to show it correctly on the display. This took more time than we expected.

RTC Switching

We wanted the system to switch to RTC immediately if the GPS signal was lost. For that, we made the RTC keep storing the latest GPS time all the time. But when the GPS came back, switching back to it smoothly wasn't that easy. Thing is DMA doesn't rearm itself automatically. So, we called it again in getLocation function. It starts DMA each second while in RTC mode.

WHY STM32 AND CUBEIDE?



High-Performance MCU:

STM32 F411RE offers excellent computational power and peripheral integration.



HAL Libraries:

Simplify I2C (for RTC) and UART (for GPS) communication.



CubeIDE Integration:

Fast peripheral setup, easy debugging, and power profiling.



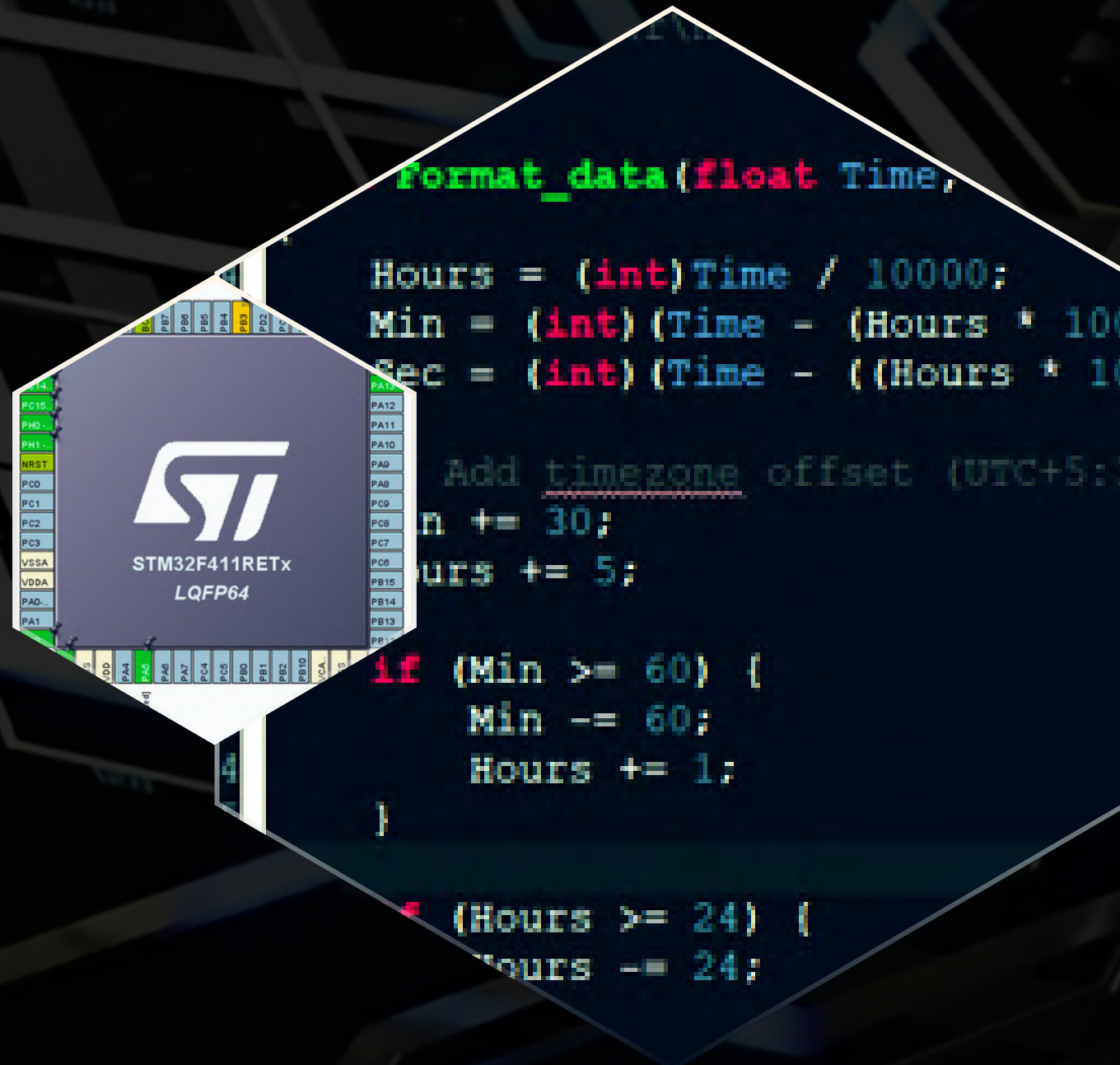
Real-Time Processing:

Suitable for parsing GPS signals and managing time-critical tasks.



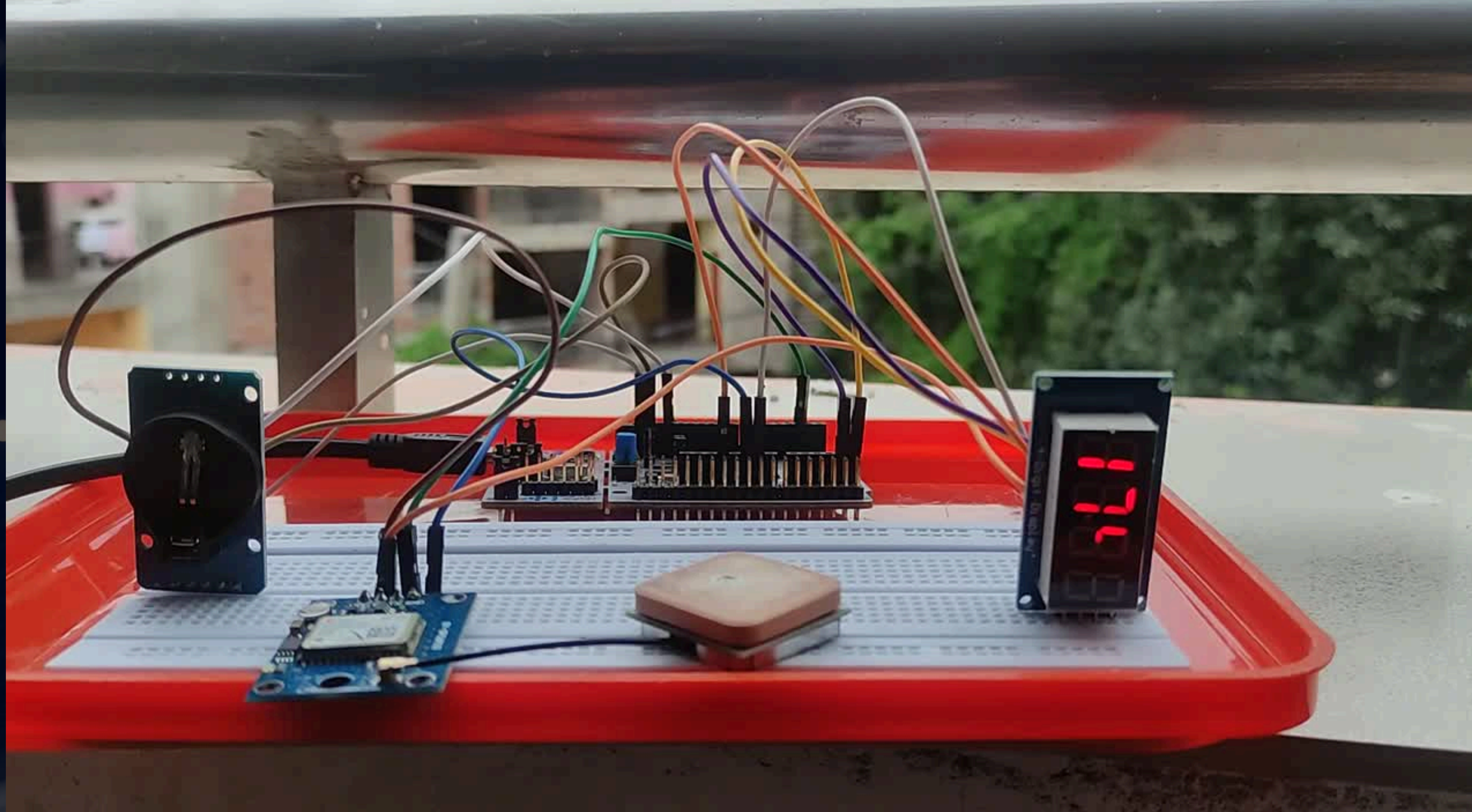
Strong Community Support:

Easy troubleshooting and extension possibilities.



DEMO VIDEO

Below is the drive link that contains all the other demo videos/photos, block diagram etc.



[https://drive.google.com/drive/folders/193muhtGHw3ECcqbhJniVB5UIBqF3IcEc?
usp=drive_linkt](https://drive.google.com/drive/folders/193muhtGHw3ECcqbhJniVB5UIBqF3IcEc?usp=drive_linkt)

FUTURE ADVANCEMENTS

- Speaker for Blind people
- Brightness auto-adjust via LDR
- Ambient temperature display (using DS18B20)
- Fault detection & alert LED for GPS loss
- Enclosure with IP65 ratings for outdoor stations
- Energy Harvesting & Backup Power – Fit integrated solar panels on the housing or rooftop mounting to trickle-charge the clock's batteries—ensuring operation even during power cuts.
- Integration with Passenger Information Systems
 - Synchronizing clocks with digital information displays and public announcement systems ensures passengers receive consistent and timely updates.
- (Reference: https://ainsit.in/Railways.aspx?utm_source=)
- Local language display on one side of the clock

CONCLUSION

The STM32-based GPS Clock with RTC fallback is a practical, scalable, and cost-effective timekeeping solution. It ensures accuracy, reliability, and portability without reliance on internet infrastructure. The project holds potential for public time displays, embedded systems, and mission-critical applications where accurate time synchronization is non-negotiable.



TEAM



Arpit Raj (2023132)

arpit23132@iiitd.ac.in



Abhinav Maurya (2023024)

abhinav23024@iiitd.ac.in



THANK YOU FOR YOUR TIME AND ATTENTION.

References:

- 1) <https://justdoelectronics.com/gps-neo-6m-module/>
- 2) <https://lastminuteengineers.com/ds3231-rtc-arduino-tutorial/>
- 3) <https://www.instructables.com/How-to-Use-the-TM1637-Digit-Display-With-Arduino/>
- 4) https://youtu.be/chI8ZoOqOY4?si=p7qgmRsKI5_QSGFX
- 5) https://youtu.be/NDJIOFYCW_s?si=yi9wNCJdXA9d2oP0
- 6) https://youtu.be/yi_i6XGZSH4?si=cYgrVs-Dyfm-EYDu
- 7) <https://youtu.be/tVBgOLjJFLE?si=luQpEOtaNmQ-LGIb>



THANK YOU!

Feel free to reach out to us.